



REPUBLIC OF YEMEN
SANA'A UNIVERSITY
FACULTY OF ENGINEERING
MECHATRONICS ENGINEERING
DEPARTMENT



DC Motor Speed Control Using PID

Done by:

Musa ALSalmani 202174202

Hasan Moqbel 202274198

Hamzah Alkhalaf 202274127

System (Parallel)

Supervised by:

Dr. Adel Rawa'a

Eng. Said Al-Samai

1. Introduction	3
2. System Identification (Z-Domain)	3
a. Modeling the Plant (DC Motor).....	3
b. Discretization	3
c. Experimental Data Collection	3
3. Open-Loop (O.L.) Analysis.....	5
a. Step Response Analysis	5
b. Frequency Response Analysis	5
4. Controller Design (Lead-Lag Compensators).....	5
a. Lead Compensator	6
b. Lag Compensator	6
c. Design Approach.....	6
5. PID Tuning Methods	8
a. Trial and Error	8
b. PID Tuner (MATLAB or Python).....	8
c. Ziegler–Nichols Method	8
6. Closed-Loop (C.L.) Analysis.....	9
a. Transient Analysis	9
b. Frequency Response	9
c. Steady-State Error.....	10
7. Implementation (AVR).....	10
a. ATmega328P Implementation	10
b. Proteus Simulation	10
c. Hardware Implementation	10
8. Conclusion.....	11

1. Introduction

DC motors are widely used in automation and control applications, and precise speed control is often required to achieve optimal performance in various systems. One of the most effective approaches for controlling the speed of a DC motor is through a Proportional-Integral-Derivative (PID) controller, which adjusts the control input based on feedback to reach the desired output speed. This report covers the steps involved in identifying the system, designing and tuning the PID controller, and implementing it on a microcontroller.

2. System Identification (Z-Domain)

To design an effective PID controller, we must first identify the transfer function of the DC motor system in the Z-domain. The following steps outline the process:

a. Modeling the Plant (DC Motor)

A DC motor can be represented by a first-order system, with the transfer function $G(s)$ in the Laplace domain defined as:

$$G(s) = \frac{K}{\tau s + 1}$$

where:

- **K** is the gain of the system.
- **τ** (tau) is the time constant of the system, which represents the motor's response time.

b. Discretization

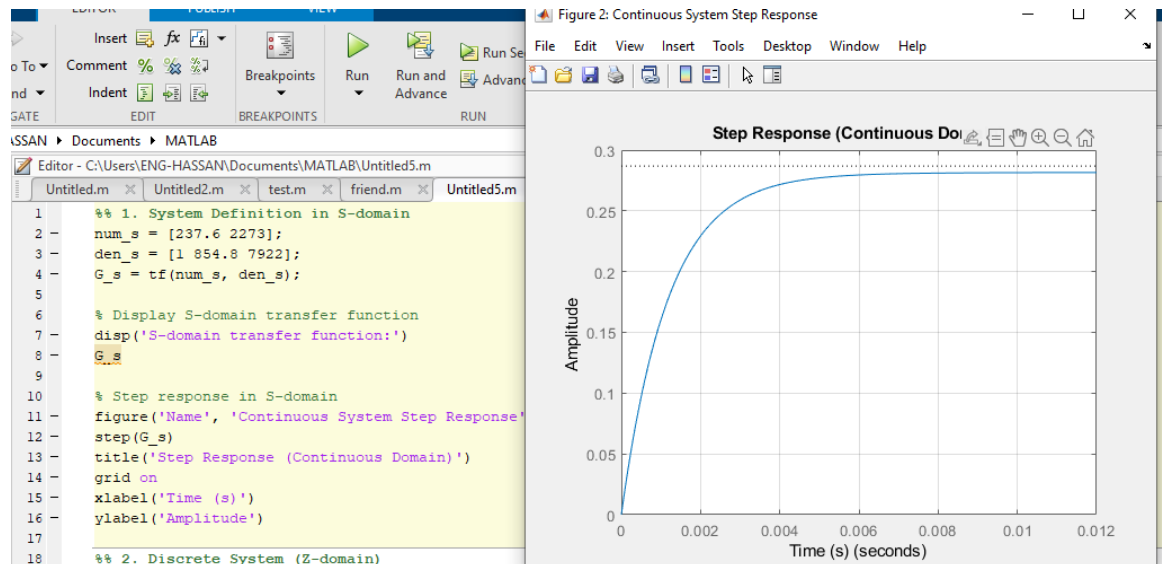
To implement a digital controller, the continuous-time transfer function $G(s)$ is converted to the discrete-time domain $G(z)$ using techniques such as Zero-Order Hold (ZOH) or Bilinear Transformation. This enables the use of digital controllers, typically implemented on microcontrollers.

c. Experimental Data Collection

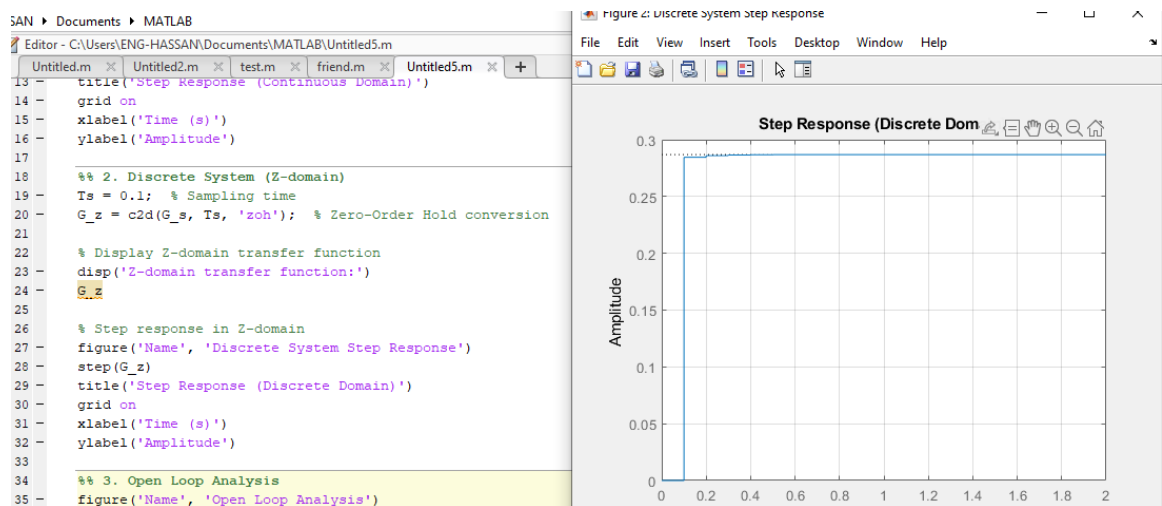
1. A step input is applied to the motor using Pulse Width Modulation (PWM).
2. The motor's response is recorded using a rotary encoder to measure the output speed.

3. System identification techniques are used to estimate the parameters K and τ , which are essential for defining the transfer function.

System Definition in S-domain



Discrete System (Z-domain)



3. Open-Loop (O.L.) Analysis

a. Step Response Analysis

The open-loop step response provides insights into the motor's performance characteristics, such as:

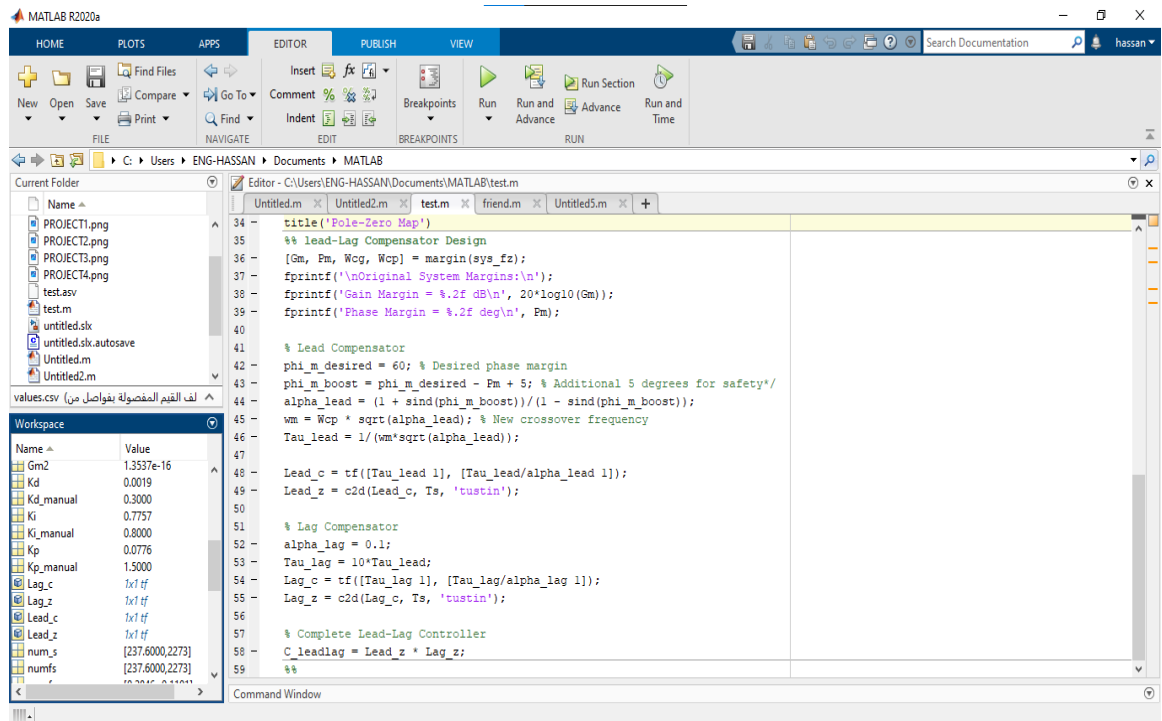
- **Settling time:** The time taken for the response to settle within a specific range.
- **Rise time:** The time taken to reach a certain percentage of the final value.
- **Steady-state error:** The final error between the desired and actual output.

b. Frequency Response Analysis

Using Bode and Nyquist plots, we analyze the system's frequency response to gain insights into:

- **Gain margin and phase margin:** These metrics are critical for assessing the system's stability in the open-loop configuration.

4. Controller Design (Lead-Lag Compensators)



```
34 - title('Pole-Zero Map')
35 - % lead-Lag Compensator Design
36 - [Gm, Fm, Wcp, Wcp] = margin(sys_tf);
37 - fprintf('\nOriginal System Margins:\n');
38 - fprintf('Gain Margin = %.2f dB\n', 20*log10(Gm));
39 - fprintf('Phase Margin = %.2f deg\n', Fm);
40 -
41 - % Lead Compensator
42 - phi_m_desired = 60; % Desired phase margin
43 - phi_m_boost = phi_m_desired - Fm + 5; % Additional 5 degrees for safety*/
44 - alpha_lead = (1 + sind(phi_m_boost))/(1 - sind(phi_m_boost));
45 - wm = Wcp * sqrt(alpha_lead); % New crossover frequency
46 - Tau_lead = 1/(wm*sqrt(alpha_lead));
47 -
48 - Lead_c = tf([Tau_lead 1], [Tau_lead/alpha_lead 1]);
49 - Lead_z = c2d(Lead_c, Ts, 'tustin');
50 -
51 - % Lag Compensator
52 - alpha_lag = 0.1;
53 - Tau_lag = 10*Tau_lead;
54 - Lag_c = tf([Tau_lag 1], [Tau_lag/alpha_lag 1]);
55 - Lag_z = c2d(Lag_c, Ts, 'tustin');
56 -
57 - % Complete Lead-Lag Controller
58 - C_leadlag = Lead_z * Lag_z;
59 - %
```

Name	Value
Gm2	1.3537e-16
Kd	0.0019
Kd_manual	0.3000
Ki	0.7757
Ki_manual	0.8000
Kp	0.0776
Kp_manual	1.5000
Lag_c	1x1 tf
Lag_z	1x1 tf
Lead_c	1x1 tf
Lead_z	1x1 tf
num_s	[237.6000, 2273]
numfs	[237.6000, 2273]

Lead-lag compensators are often used alongside PID controllers to refine system behavior.

a. Lead Compensator

The lead compensator improves the phase margin and transient response of the system. Its transfer function is given by:

$$D_{lead}(s) = K_c \frac{\tau_1 s + 1}{\alpha \tau_1 s + 1}, \quad \alpha < 1$$

where τ_1 and α are design parameters.

b. Lag Compensator

The lag compensator enhances steady-state accuracy by increasing the system gain. The transfer function for a lag compensator is:

$$D_{lag}(s) = K_c \frac{\alpha \tau_2 s + 1}{\tau_2 s + 1}, \quad \alpha > 1$$

where τ_2 and α are parameters chosen to improve steady-state error.

c. Design Approach

The root locus or frequency domain methods can be used to design the compensators to meet specified transient and steady-state performance criteria.

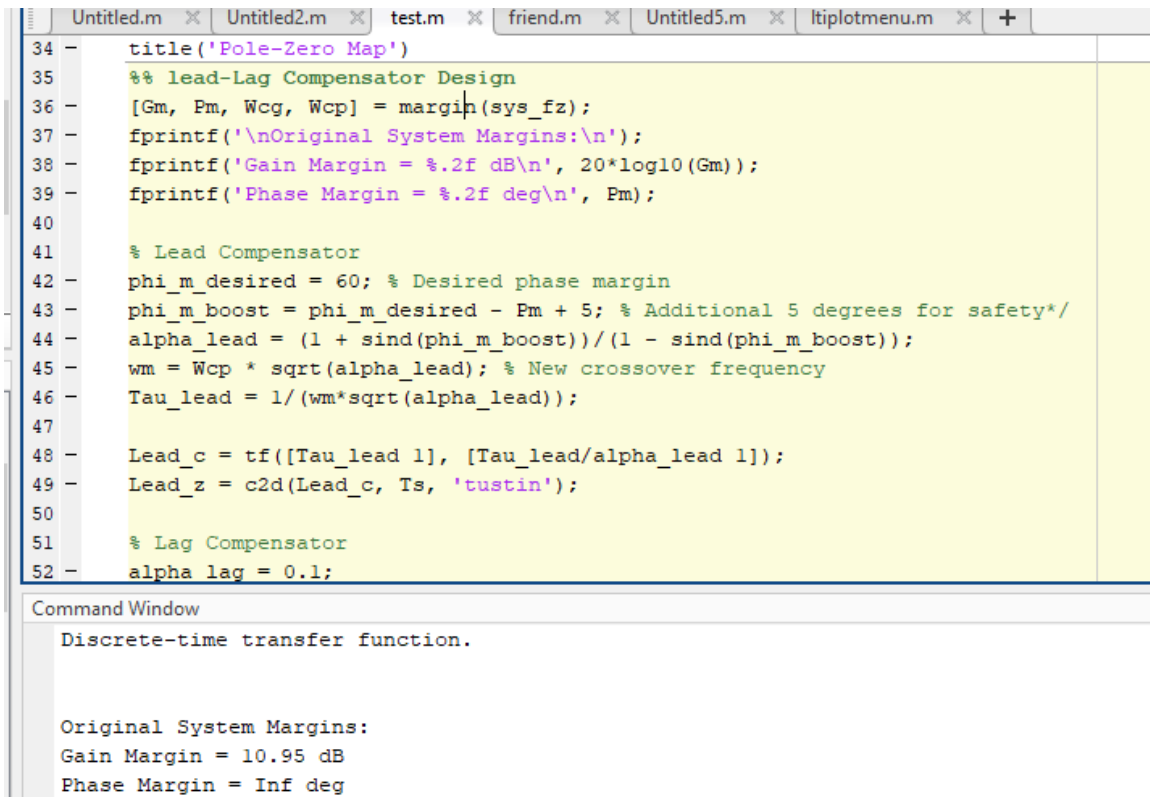
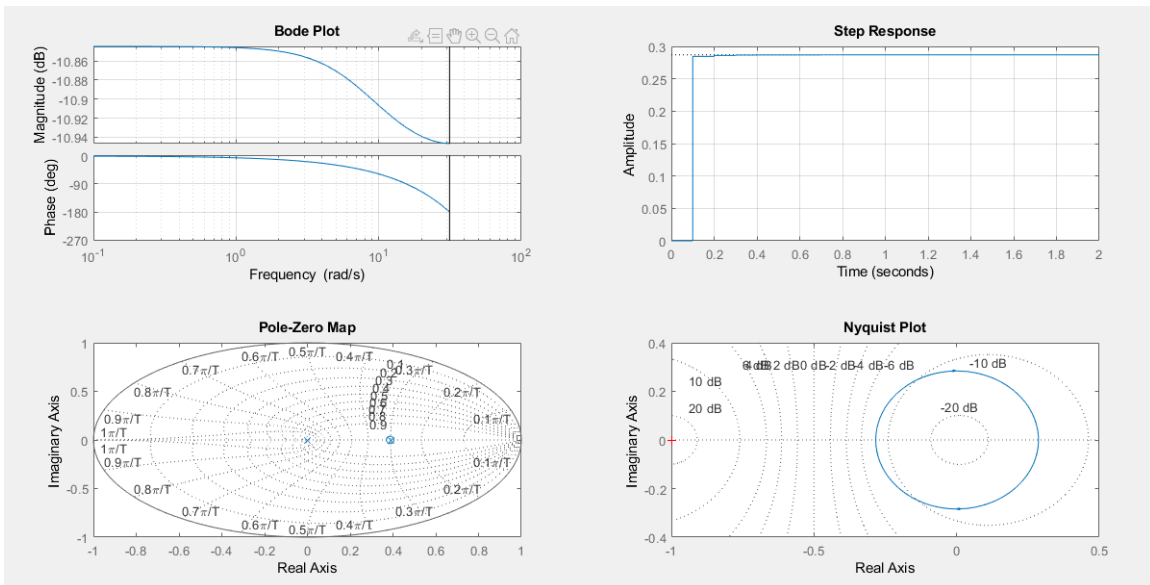


Figure of an Open Loop Analysis



5. PID Tuning Methods

After identifying the plant and designing initial compensators, we proceed with PID tuning to refine the controller.

a. Trial and Error

The controller parameters K_p , K_i , and K_d are adjusted manually to achieve the desired system performance. This approach requires repeated testing and observation.

b. PID Tuner (MATLAB or Python)

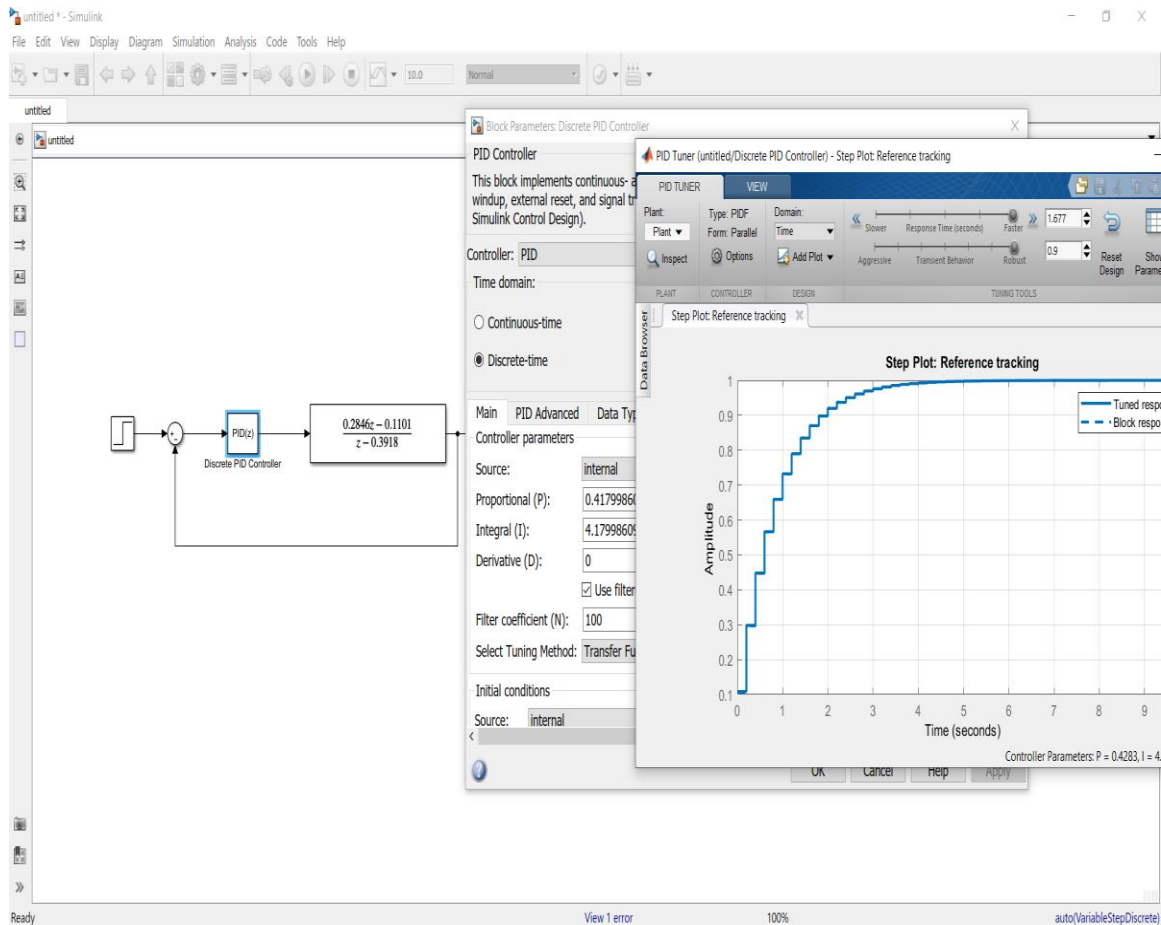
Tools like MATLAB or Python's control libraries can be used to automate the tuning process. These tools help optimize the parameters based on performance criteria like settling time and overshoot.

c. Ziegler–Nichols Method

The Ziegler–Nichols method provides an empirical approach to tuning. The steps include:

1. Identifying the ultimate gain K_u and ultimate period T_u by increasing K until sustained oscillations are observed.

Using K_u and T to calculate K_p , K_i , and K_d values based on Ziegler–Nichols formulas for a PID controller.



6. Closed-Loop (C.L.) Analysis

a. Transient Analysis

With the PID controller in place, the system's transient performance is analyzed:

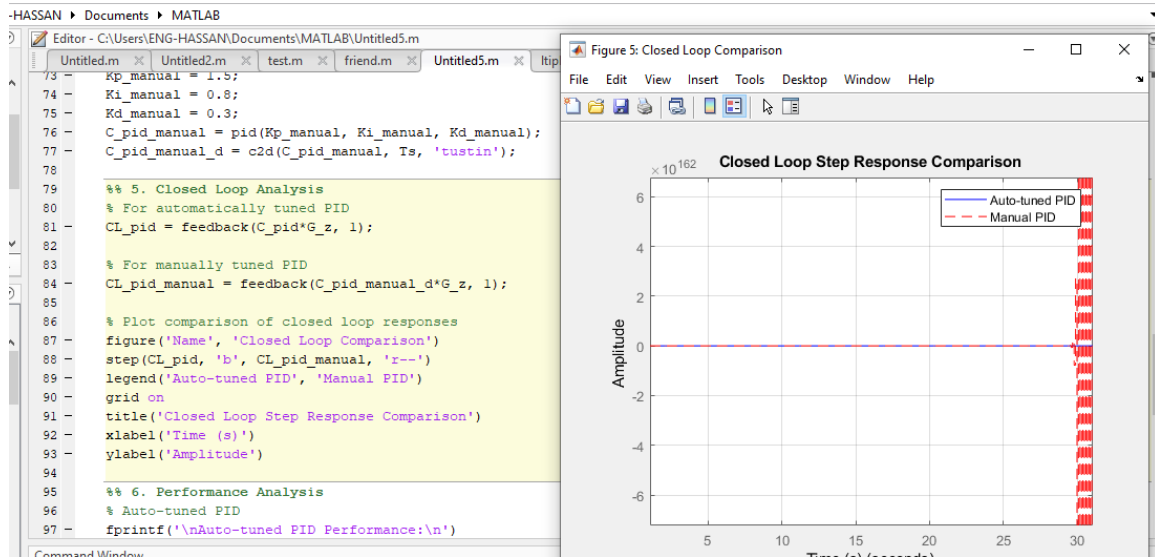
- Settling time • Overshoot • Rise time

b. Frequency Response

We assess the closed-loop system's gain and phase margins to verify stability and robustness.

c. Steady-State Error

To ensure accurate speed tracking, the system's steady-state error is evaluated for various setpoints.



7. Implementation (AVR)

To implement the PID controller, we selected the ATmega328P microcontroller, using the following steps:

a. ATmega328P Implementation

The PID control algorithm is developed in C and executed in real-time using timer interrupts to maintain a fixed sampling period.

b. Proteus Simulation

The entire system, including the motor, sensors, and controller, is simulated in Proteus to validate the design and ensure proper communication and control performance.

c. Hardware Implementation

The control system is constructed using the ATmega328P, ESP32, L298N motor driver, and rotary encoder. Testing is conducted on the actual hardware, with parameters adjusted as necessary to ensure effective speed control.

AVR

// PID implementation

float Kp = 1.2;

float Ki = 0.8;

float Kd = 0.3;

float previous_error = 0;

float integral = 0;

float PID_control(float setpoint, float measured_value) {

 float error = setpoint - measured_value;

 integral += error * dt;

 float derivative = (error - previous_error) / dt;

 float output = Kp * error + Ki * integral + Kd * derivative;

 previous_error = error;

 return output;

}

8. Conclusion

A PID controller offers effective speed control for a DC motor, and tuning methods like Ziegler–Nichols and trial-and-error enable optimized performance. Lead and lag compensators further enhance transient and steady-state performance. Through simulation and hardware testing, we achieve robust and stable motor speed control, meeting both performance and implementation requirements.

Import - C:\Users\ENG-HASSAN\Documents\MATLAB\values.csv

IMPORT VIEW

Delimited Column delimiters: Comma Range: C24:C24 Output Type: Column vectors

Fixed Width Delimiter Opti... Variable Names Row: 1 Text Options

Replace unimportable cells with NaN

Import Selection

values.csv

	A sampleNo Number	B input Number	C output Number
1	1.07	100	0.00
2	1.17	100	6.00
3	1.27	100	21.00
4	1.37	100	27.00
5	1.47	100	27.00
6	1.57	100	27.00
7	1.67	100	30.00
8	1.77	100	27.00
9	1.87	100	30.00
10	1.97	100	27.00
11	2.07	100	30.00
12	2.17	100	30.00
13	2.27	100	27.00
14	2.37	100	30.00
15	2.48	100	27.00
16	2.58	100	30.00
17	2.68	100	27.00
18	2.78	100	30.00
19	2.88	100	27.00
20	2.98	100	30.00
21	3.08	100	27.00
22	3.18	100	30.00
23	3.28	100	27.00
24	3.38	100	30.00

Editor - C:\Users\ENG-HASSAN\Documents\MATLAB\test.m

Untitled.m x Untitled2.m x test.m x +

```

1 - numfs=[237.6 2273];
2 - denfs=[1 854.8 7922];
3 - sys_fs= tf(numfs,denfs)
4 - %%
5 - z=tf('z');
6 - sys_fs=(237.6*z + 2273)/(z+854.8*z+7922)

```

Command Window

```

sys_fs =

      237.6 s + 2273
-----
      s^2 + 854.8 s + 7922

Continuous-time transfer function.

sys_fs =

      237.6 z + 2273
-----
      855.8 z + 7922

Sample time: unspecified
Discrete-time transfer function.

```

>>

MATLAB R2020a

HOME PLOTS APPS EDITOR PUBLISH VIEW

Find Files Find Go To Comment % % % Breakpoints Run Run and Advance Run and Time

FILE NAVIGATE EDIT BREAKPOINTS RUN

C:\Users\ENG-HASSAN\Documents\MATLAB

Current Folder

Workspace

Name	Value
alpha_lag	0.1000
alpha_lead	NaN
ans	1x1 struct
C_leadlag	1x1 tf
C_pid	1x1 pid
C_pid_manual	1x1 pid
C_pid_manual_d	1x1 pid
CL_pid	1x1 tf
CL_pid_manual	1x1 tf
den_s	[1,854.8000,7922]
denfs	[1,854.8000,7922]
denfz	[1,-0.3918,0]
G_s	1x1 tf

values.csv (لف القيم المفصلة بفواصل من)

Editor - C:\Users\ENG-HASSAN\Documents\MATLAB\test.m

```

1 - numfs=[237.6 2273];
2 - denfs=[1 854.8 7922];
3 - sys_fs=tf(numfs,denfs)
4 - Ts=0.1;
5 - sys_f=c2d(sys_fs,Ts,'zoh')
6 - %%
7 - numfz=[0.2846 -0.1101];
8 - denfz=[1 -0.3918 0];
9 - sys_fz=tf(numfz,denfz)
10 - z=tf('z');
11 - sys_fz=(0.2846*z - 0.1101)/(z^2 - 0.3918*z+0)
12 - %%
13 - step(sys_fz)
14 - title('Step Response')
15 - grid on
16 - xlabel('Time (s)')
17 - ylabel('Amplitude')
18 - %% root locus
19 - rlocus(sys_fz)
20 - axis([-1 1 -1 1])
21 - zeta = 4.8;
22 - Wn= 89.1;
23 - zgrid(zeta,Wn)
24 -
25 - %%
26 - % Bode Plot

```

Command Window

UTF-8 script Ln 51 Col 6

MATLAB R2020a

HOME PLOTS APPS EDITOR PUBLISH VIEW

Find Files Find Go To Comment % % % Breakpoints Run Run and Advance Run and Time

FILE NAVIGATE EDIT BREAKPOINTS RUN

C:\Users\ENG-HASSAN\Documents\MATLAB

Current Folder

Workspace

Name	Value
alpha_lag	0.1000
alpha_lead	NaN
ans	1x1 struct
C_leadlag	1x1 tf
C_pid	1x1 pid
C_pid_manual	1x1 pid
C_pid_manual_d	1x1 pid
CL_pid	1x1 tf
CL_pid_manual	1x1 tf
den_s	[1,854.8000,7922]
denfs	[1,854.8000,7922]
denfz	[1,-0.3918,0]
G_s	1x1 tf

values.csv (لف القيم المفصلة بفواصل من)

Editor - C:\Users\ENG-HASSAN\Documents\MATLAB\test.m

```

26 - % Bode Plot
27 - bode(sys_fz)
28 - grid on
29 - title('Bode Plot')
30 - %%
31 - % Pole-Zero Map
32 - pzmap(sys_fz)
33 - grid on
34 - title('Pole-Zero Map')
35 - %% lead-Lag Compensator Design
36 - [Gm, Pm, Wcg, Wcp] = margin(sys_fz);
37 - fprintf('\nOriginal System Margins:\n');
38 - fprintf('Gain Margin = %.2f dB\n', 20*log10(Gm));
39 - fprintf('Phase Margin = %.2f deg\n', Pm);
40 -
41 - % Lead Compensator
42 - phi_m_desired = 60; % Desired phase margin
43 - phi_m_boost = phi_m_desired - Pm + 5; % Additional 5 degrees for safety*
44 - alpha_lead = (1 + sind(phi_m_boost))/(1 - sind(phi_m_boost));
45 - wm = Wcp * sqrt(alpha_lead); % New crossover frequency
46 - Tau_lead = 1/(wm*sqrt(alpha_lead));
47 -
48 - Lead_c = tf([Tau_lead 1], [Tau_lead/alpha_lead 1]);
49 - Lead_z = c2d(Lead_c, Ts, 'tustin');
50 -
51 - % Lag Compensator

```

Command Window

UTF-8 script Ln 33 Col 8

